

DAY 23

Webhooks and Real-Time Automation

Event-driven AI automation with n8n, APIs, security, reliability, and real business workflows

Course: GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Duration: 60-minute lecture + practical mini project

Level: Beginner to Intermediate

DAY 23 CORE PRINCIPLE

A webhook turns an event into an immediate workflow trigger. A production webhook system must verify the sender, validate the payload, prevent duplicate side effects, acknowledge quickly, and process the event safely.

Table of Contents

1. Day 23 Goals and 60-Minute Schedule
 2. Day 22 Review: APIs Before Webhooks
 3. What Is a Webhook?
 4. API vs Polling vs Webhook
 5. Event-Driven Architecture
 6. Anatomy of a Webhook Request
 7. Webhook Lifecycle and Delivery
 8. Receiving Webhooks in Python
 9. Security: Authentication and Signatures
 10. Reliability: Retries, Duplicates, and Idempotency
 11. Fast Acknowledgment and Asynchronous Processing
 12. n8n Webhook Node in Practice
 13. n8n Response Patterns
 14. Transforming and Routing Webhook Data
 15. AI Inside a Webhook Workflow
 16. Advanced Example: OpenAI Webhook Events
 17. Error Handling, Logging, and Observability
 18. Mini Project: Real-Time Lead Intake Automation
 19. Testing Strategy and Failure Scenarios
 20. Business Use Cases and Architecture Patterns
 21. Quiz, Exercises, and Homework
 22. Interview and Outsourcing Preparation
 23. Day 23 Checklist and Day 24 Preview
- References and Further Reading

HOW TO USE THIS PDF

Teach the concepts first, demonstrate one webhook end-to-end, then let students build the lead-intake project. The code and n8n patterns are intentionally modular so they can be reused in client projects.

1. Day 23 Goals and 60-Minute Schedule

Day 23 moves the course from request-driven API calls to event-driven automation. Instead of asking another system repeatedly whether something changed, our workflow receives an event when the change happens.

Learning objectives

- Explain what a webhook is and why it is useful.
- Compare webhooks, polling, and normal API calls.
- Read a webhook HTTP request: method, URL, headers, and JSON body.
- Build a webhook receiver in Python and n8n.
- Validate event payloads before using them.
- Protect webhook endpoints with authentication or signature verification.
- Handle retries and duplicate deliveries with idempotency.
- Choose between synchronous and asynchronous processing.
- Connect webhook events to GPT, APIs, CRM systems, databases, and notifications.
- Design and test a production-ready event-driven workflow.

60-minute lecture schedule

Time	Topic
0-5 min	Day 22 API review
5-12 min	Webhook fundamentals and event-driven architecture
12-20 min	HTTP payloads, headers, and lifecycle
20-28 min	Security and signature verification
28-35 min	Retries, duplicates, idempotency, and fast acknowledgment
35-44 min	n8n Webhook node and response patterns
44-52 min	AI + webhook business automation
52-57 min	Mini project build
57-60 min	Quiz, recap, homework

2. Day 22 Review: APIs Before Webhooks

Day 22 taught how an application actively calls an API. Webhooks reverse the direction for event notifications.

API CALL

Your Workflow

|

| HTTP request: GET /orders/A123

v

External API

|

| JSON response

v

Your Workflow

API concepts that still matter

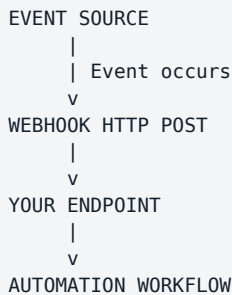
- HTTP methods such as GET and POST.
- URLs and endpoints.
- Headers such as Content-Type and Authorization.
- JSON request and response bodies.
- HTTP status codes.
- Authentication.
- Rate limits and retries.
- Error handling and logging.

BRIDGE FROM DAY 22 TO DAY 23

An API lets your workflow ask another system for information or perform an action. A webhook lets another system notify your workflow that an event occurred.

3. What Is a Webhook?

Beginner definition: A webhook is an HTTP endpoint that receives data automatically when a specific event occurs in another system.



Example

A customer submits a lead form. The form platform immediately sends a JSON payload to your n8n webhook URL. n8n starts the workflow, validates the lead, calls AI to classify it, saves it to a CRM, and notifies sales.

Common webhook events

- New form submission
- Payment completed
- Order created
- Order fulfilled
- Support ticket created
- CRM contact updated
- Git push or pull request
- Calendar booking created
- Background AI job completed

Why businesses use webhooks

They reduce polling, decrease delay, and let workflows react close to the time an event happens. This makes them a core integration mechanism for modern automation.

4. API vs Polling vs Webhook

Pattern	Who initiates?	Timing	Best use
API request	Your application	When your app asks	Fetch data or perform an action
Polling	Your application repeatedly	Every N seconds/minutes	Fallback when no webhook exists
Webhook	Event source	When event occurs	Real-time event notification

Polling example

Every 5 minutes:

GET /orders?updated_since=...

No changes?
Wait 5 minutes.
Ask again.

Webhook example

Order is created
|
v
POST https://automation.example.com/webhook/order-created
|
v
Workflow starts immediately

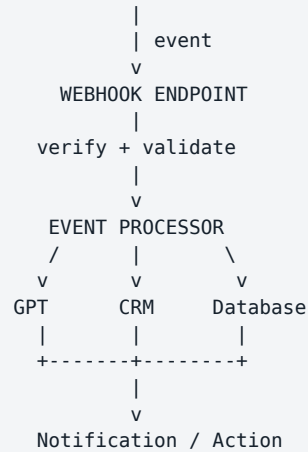
DESIGN RULE

Prefer a reliable webhook when the source system supports the event you need. Use polling when events are unavailable, unreliable, or when periodic reconciliation is still required.

5. Event-Driven Architecture

A webhook is one component of a larger event-driven architecture. The important design idea is that business events trigger downstream work.

Customer / SaaS / Payment / CRM



Event vs command

Type	Example	Meaning
Event	payment.completed	Something already happened
Command	issue_refund	Please perform an action

Treating events as facts helps prevent architectural confusion. A webhook saying payment.completed should normally be recorded and processed; it is not itself permission to perform unrelated sensitive actions.

6. Anatomy of a Webhook Request

Most webhook deliveries are HTTP requests, commonly POST requests with JSON bodies.

```
POST /webhooks/lead HTTP/1.1
Host: automation.example.com
Content-Type: application/json
X-Webhook-Signature: abc123...
X-Event-ID: evt_9001

{
  "event": "lead.created",
  "created_at": "2026-08-19T04:10:00Z",
  "data": {
    "name": "Ava Chen",
    "email": "ava@example.com",
    "message": "We need an AI support assistant."
  }
}
```

Four areas to inspect

Area	What it tells you
Method	How the sender is calling the endpoint
URL/path	Which webhook or event route is targeted
Headers	Content type, authentication, signature, event ID, timestamps
Body	The actual event payload, usually JSON

Do not trust the JSON just because it arrived

Treat every inbound payload as untrusted external input. Validate required fields, expected types, size, event type, and authorization before creating side effects.

7. Webhook Lifecycle and Delivery

1. Register webhook URL
- |
2. Event occurs in source system
- |
3. Source creates event payload
- |
4. Source sends HTTP request
- |
5. Receiver verifies sender
- |
6. Receiver validates payload
- |
7. Receiver deduplicates event
- |
8. Receiver acknowledges receipt
- |
9. Business processing runs
- |
10. Logs and outcome are recorded

Registration

Many SaaS platforms let you paste a callback URL into a dashboard or create the subscription using an API. You typically choose which event types should be delivered.

Acknowledgment

The receiver normally returns a successful 2xx HTTP status to confirm receipt. A non-success response may cause the provider to retry, depending on the provider.

Important reality

Webhook systems are usually designed for at-least-once delivery rather than exactly-once processing. Your application must be prepared to see the same event more than once.

8. Receiving Webhooks in Python

The following simplified Flask example shows the mechanics of receiving a JSON webhook. It is intentionally incomplete from a security perspective; signature verification is added in the next section.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.post("/webhooks/lead")
def lead_webhook():
    payload = request.get_json(silent=True)

    if not payload:
        return jsonify({"error": "invalid_json"}), 400

    event_type = payload.get("event")
    data = payload.get("data", {})

    if event_type != "lead.created":
        return jsonify({"status": "ignored"}), 200

    if not data.get("email"):
        return jsonify({"error": "missing_email"}), 400

    print("Received lead:", data)

    return jsonify({"received": True}), 200

if __name__ == "__main__":
    app.run(port=8000)
```

What this example teaches

- Create a public route.
- Accept the correct HTTP method.
- Parse JSON.
- Validate event type and required fields.
- Return an HTTP response.

PRODUCTION WARNING

A public endpoint should also implement sender verification, payload size limits, replay protection where applicable, idempotency, safe logging, and authorization boundaries.

9. Security: Authentication and Signatures

Webhook URLs are public integration endpoints. Security must be designed, not assumed.

Common protection mechanisms

Mechanism	Purpose	Notes
Basic authentication	Require username/password	Simple; supported by some automation tools
Header token	Require secret header	Useful for controlled integrations
JWT authentication	Validate signed token	Useful when sender supports JWT
Webhook signature	Verify message cryptographically	Preferred when provider supplies signing secret
IP allowlist	Limit network sources	Supplementary; provider IPs may change

Current n8n Webhook credentials support Basic auth, Header auth, and JWT auth for services calling the webhook URL. [1]

Signature-verification model

```

Sender
  |
  | payload + timestamp + signature
  v
Receiver
  |
  | recompute/verify signature
  v
Valid? ---- no ----> reject
  | yes
  v
process event

```

Security rules

- Use HTTPS.
- Keep signing secrets in environment variables or credential stores.
- Verify signatures before parsing into business actions when the provider requires raw body verification.
- Reject events with invalid signatures.
- Protect against replay with timestamps/nonces when supported.
- Rotate exposed secrets.
- Do not log secrets or sensitive full payloads unnecessarily.

10. Reliability: Retries, Duplicates, and Idempotency

Why duplicates happen

The sender may deliver an event, fail to receive your acknowledgment, and retry. The first delivery may actually have been processed successfully. That can produce duplicates.

```
Delivery #1
Webhook -> process payment event -> response lost

Sender thinks delivery failed

Delivery #2
Same webhook event arrives again
```

Dangerous result without idempotency

```
evt_1001 -> create invoice
evt_1001 -> create invoice again

Result: duplicate invoice
```

Idempotent pattern

```
event_id = "evt_1001"

if event_id already_processed:
    return 200

process_event()
mark_processed(event_id)
return 200
```

Idempotency means designing processing so duplicate delivery does not create duplicate business side effects.

Store for deduplication

Typical keys include provider event ID, webhook ID, transaction ID, or a deterministic idempotency key created from stable event fields.

11. Fast Acknowledgment and Asynchronous Processing

A frequent production mistake is keeping the webhook HTTP connection open while performing long AI calls, database operations, or multiple external API requests.

Safer event pattern

```
Webhook received
|
v
Verify signature
|
Validate + dedupe
|
Persist / enqueue event
|
Return 2xx quickly
|
v
Background processing
- AI classification
- CRM lookup
- database writes
- notifications
```

When synchronous processing is appropriate

If you intentionally expose an HTTP endpoint where the caller needs the result of the workflow, synchronous response may be correct. Example: a website calls your endpoint and waits for a generated summary.

When asynchronous processing is better

If the incoming request is an event notification and the sender mainly needs confirmation of receipt, acknowledge quickly and process longer work separately.

RELIABILITY PRINCIPLE

Separate delivery acknowledgment from business completion. Receiving an event successfully does not necessarily mean the entire business workflow has finished.

12. n8n Webhook Node in Practice

n8n provides a Webhook trigger node that exposes an HTTP endpoint and starts a workflow when a request arrives. Current n8n documentation provides separate Test and Production URLs. The Test URL is used during workflow development; the Production URL is used after the workflow is published/active. [1][2]

Basic n8n flow

```

Webhook
  |
  v
Edit Fields / Set
  |
  v
IF / Switch
  |
  +-----> ignored event
  |
  v
AI / HTTP Request / CRM
  |
  v
Database / Notification

```

Webhook node setup checklist

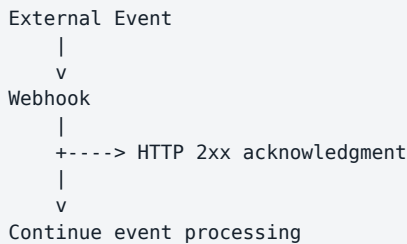
- Choose the HTTP method expected from the sender.
- Choose a unique path such as /lead-created or /support-event.
- Use the Test URL while designing the workflow.
- Send sample requests and inspect the incoming headers/body/query parameters.
- Add authentication when supported by the sender.
- Switch the external service to the Production URL when ready.
- Publish/activate the workflow according to the current n8n workflow lifecycle.

n8n authentication

Current n8n Webhook documentation supports Basic auth, Header auth, and JWT auth. [1]

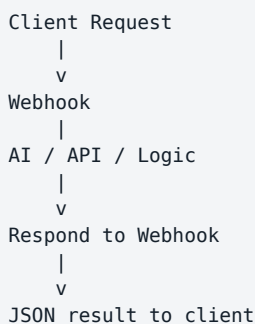
13. n8n Response Patterns

Pattern A - Immediate event acknowledgment



Use when the caller mainly needs confirmation that the event was received.

Pattern B - Workflow result returned to caller



n8n has a dedicated Respond to Webhook node for controlling the response when a workflow needs to return a specific body/status/headers. [3]

Example JSON response

```

{
  "accepted": true,
  "lead_id": "LEAD-2041",
  "classification": "qualified"
}
  
```

Design question

Ask: Does the sender need a business result, or only delivery confirmation? That determines whether your webhook should behave like an event receiver or a synchronous API endpoint.

14. Transforming and Routing Webhook Data

Incoming webhook payloads are rarely in the exact structure your downstream systems need.

Normalize first

Incoming payload

```
{
  "customer_email": "ava@example.com",
  "msg": "Need support automation",
  "source": "website"
}
```

Normalized internal event

```
{
  "email": "ava@example.com",
  "message": "Need support automation",
  "source": "website",
  "received_at": "..."
}
```

Then route

```
Webhook
|
Normalize
|
Validate
|
Switch on event_type
/   |   \
v   v   v
Lead Order Support
|   |   |
CRM  ERP Ticketing
```

Why create an internal schema?

- Downstream nodes become simpler.
- Vendor-specific field names are isolated at the boundary.
- Validation becomes consistent.
- Multiple event sources can feed the same business workflow.
- Testing is easier.

15. AI Inside a Webhook Workflow

AI is useful after the webhook has passed basic security and validation. The model should interpret language, extract information, classify, summarize, or recommend - not decide whether the sender is trusted.

Example lead workflow

```

Website Lead Event
  |
  v
Webhook
  |
  v
Verify + Validate
  |
  v
GPT structured classification
  |
  +--> intent
  +--> company
  +--> business need
  +--> urgency
  +--> missing fields
  |
  v
Deterministic scoring
  |
  v
CRM + Slack + follow-up queue
  
```

Structured AI output

```

{
  "summary": "Prospect wants AI customer support automation.",
  "intent": "sales",
  "service": "ai_automation",
  "urgency": "high",
  "missing_information": ["budget"],
  "requires_human_review": true
}
  
```

Important division of responsibility

Task	Owner
Verify webhook signature	Application / platform
Validate JSON schema	Application / n8n
Interpret free-text message	GPT
Lead score threshold	Code / workflow rule
Write CRM record	CRM integration
Approve sensitive outbound action	Business rule / human

16. Advanced Example: OpenAI Webhook Events

OpenAI currently supports project-level webhook endpoints for real-time API events such as background response completion. Webhooks are sent to a public HTTP endpoint and include a signing secret for verification. [4]

Current OpenAI delivery guidance

The current OpenAI webhook guide recommends returning a successful 2xx response quickly, offloading non-trivial processing, and verifying webhook signatures. The guide also notes retries on unsuccessful/slow delivery and recommends using the webhook ID for deduplication because duplicate events may occur. [4]

Python verification example - simplified from current SDK pattern

```
import os
from flask import Flask, request, Response
from openai import OpenAI, InvalidWebhookSignatureError

app = Flask(__name__)
client = OpenAI(
    webhook_secret=os.environ["OPENAI_WEBHOOK_SECRET"]
)

@app.post("/openai-webhook")
def openai_webhook():
    try:
        event = client.webhooks.unwrap(
            request.data,
            request.headers
        )

        if event.type == "response.completed":
            response_id = event.data.id
            print("Completed:", response_id)

        return Response(status=200)

    except InvalidWebhookSignatureError:
        return Response("Invalid signature", status=400)
```

When signature verification depends on the raw request body, do not transform the body before verification. [4]

17. Error Handling, Logging, and Observability

Three layers of failure

Layer	Example	Response
Delivery	Endpoint unavailable	Provider retry / alert
Validation	Missing event ID	Reject or quarantine
Business processing	CRM API fails	Retry job / dead-letter / human review

What to log

- Event ID and event type.
- Received timestamp.
- Verification result.
- Validation result.
- Idempotency/deduplication result.
- Workflow execution ID.
- Downstream API results - without leaking secrets.
- Final business status: completed, retrying, failed, escalated.

Do not log blindly

Webhook bodies may contain personal, financial, authentication, or confidential data. Log only what is needed for operations and debugging.

Operational statuses

```
RECEIVED
-> VERIFIED
-> VALIDATED
-> DEDUPED
-> ACKNOWLEDGED
-> PROCESSING
-> COMPLETED

or

-> RETRY_PENDING / FAILED / HUMAN_REVIEW
```

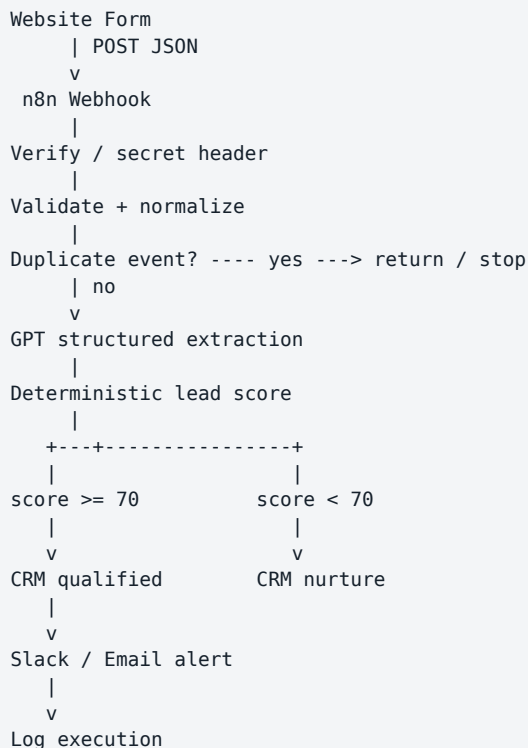
18. Mini Project: Real-Time Lead Intake Automation

Project goal: Build a real-time inbound sales lead workflow using n8n, GPT, a CRM/database destination, and a human notification channel.

Business scenario

A consulting company receives website leads. It wants every submission processed instantly, classified by AI, scored using deterministic rules, saved to CRM, and routed to sales if qualified.

Architecture



18A. Mini Project - Input Contract

Example webhook payload

```
{
  "event_id": "lead_evt_2041",
  "event": "lead.created",
  "data": {
    "name": "Ava Chen",
    "email": "ava@northstar.example",
    "company": "Northstar Retail",
    "employees": 120,
    "message": "We need AI support automation for Shopify and email.",
    "budget_range": "10k-25k",
    "source": "website"
  }
}
```

Validation rules

Field	Rule
event_id	Required, non-empty string
event	Must equal lead.created
data.email	Required and syntactically plausible
data.message	Required; maximum allowed length
employees	Optional number; reject impossible values
budget_range	Optional controlled value

Normalized internal object

```
{
  "lead_id": "lead_evt_2041",
  "email": "ava@northstar.example",
  "company": "Northstar Retail",
  "employees": 120,
  "message": "We need AI support automation for Shopify and email.",
  "budget_range": "10k-25k",
  "source": "website"
}
```

18B. Mini Project - AI Extraction

AI job

Use GPT for language interpretation only. Request a structured result that downstream nodes can validate.

SYSTEM / INSTRUCTIONS

You are a B2B lead-analysis component.

Extract only information supported by the lead message and provided fields. Do not invent company size, budget, urgency, or technical requirements.

Return structured fields:

- summary
- business_problem
- requested_capabilities[]
- buying_intent: low | medium | high
- urgency: low | medium | high
- missing_information[]
- requires_human_review

Example result

```
{
  "summary": "Retail company wants AI customer support automation.",
  "business_problem": "manual support workload",
  "requested_capabilities": ["Shopify", "email automation"],
  "buying_intent": "high",
  "urgency": "medium",
  "missing_information": ["timeline"],
  "requires_human_review": false
}
```

AI BOUNDARY

The model may classify intent and interpret the message. It should not invent a budget or decide a binding commercial commitment.

18C. Mini Project - Deterministic Lead Score

Lead scoring is easier to audit when the numeric score is calculated outside the LLM.

```
score = 0

if employees >= 50:
    score += 20

if budget_range in ["10k-25k", "25k+"]:
    score += 30

if buying_intent == "high":
    score += 30

if "AI" in message or "automation" in message.lower():
    score += 20

qualified = score >= 70
```

Routing table

Score	Route	Action
70-100	Qualified	Create CRM opportunity + notify sales
40-69	Warm	Create lead + nurture follow-up
0-39	Low priority	Store lead; no urgent notification

Why deterministic?

Business stakeholders can review the weights, change thresholds, and audit why a lead was classified as qualified.

18D. Mini Project - Suggested n8n Node Sequence

#	Node	Purpose
1	Webhook	Receive lead.created
2	Edit Fields / Set	Normalize payload
3	IF	Validate event and required fields
4	Database / Data Store	Check event_id for duplicate
5	OpenAI / AI node	Extract structured lead intelligence
6	Code / Edit Fields	Compute deterministic score
7	IF / Switch	Route by score
8	CRM / HTTP Request	Create/update lead or opportunity
9	Slack / Email	Notify sales for high-priority leads
10	Database	Record processing status

Optional response

```
{
  "accepted": true,
  "event_id": "lead_evt_2041"
}
```

For a real event webhook, consider acknowledging before long downstream work. For a synchronous endpoint intentionally built to return lead analysis, use a response node after processing.

19. Testing Strategy and Failure Scenarios

Test matrix

Test	Expected result
Valid lead	Accepted; processed once
Missing email	Validation failure
Wrong event type	Ignored or routed elsewhere
Invalid secret/signature	Rejected
Duplicate event ID	No duplicate CRM side effect
AI API timeout	Retry or queue; do not lose event
CRM API failure	Retry/escalate; keep event status
Very long message	Reject/trim according to policy
Prompt injection in message	Treat as untrusted lead text; system rules remain authoritative
Sales notification failure	CRM record remains; notification retry

Local testing

A webhook receiver generally needs a reachable URL. n8n provides test webhook URLs during workflow development. For local custom servers, developers commonly use secure tunneling or cloud development environments to expose a temporary public endpoint.

Regression principle

After changing validation, prompts, routing logic, or integrations, rerun the whole test set. Do not test only the new scenario.

20. Business Use Cases and Architecture Patterns

Customer support

Ticket Created -> Webhook -> GPT Triage -> RAG -> Assign Queue -> Notify

Ecommerce

Order Created -> Webhook -> Validate -> Fraud/Rules -> CRM/ERP -> Customer Workflow

Finance operations

Invoice Event -> Webhook -> Extract/Validate -> Accounting -> Approval -> Audit Log

Sales

Lead Submitted -> Webhook -> AI Qualification -> CRM -> Sales Alert -> Follow-up

AI background jobs

Long AI Job -> completion event -> Webhook -> verify -> retrieve result -> save -> notify

Developer operations

Repository Event -> Webhook -> CI / Review Workflow -> Slack / Ticket

Pattern selection

Need	Pattern
Immediate event reaction	Webhook trigger
Need current data on demand	API request
No event support	Polling
Need immediate receipt + slow processing	Webhook + queue/background
Need caller to receive final answer	Synchronous webhook/API endpoint

21. Quiz, Exercises, and Homework

10-question quiz

#	Question	Answer
1	What starts a webhook workflow?	An event causes an HTTP request to the registered endpoint.
2	Webhook vs polling?	Webhook pushes on events; polling repeatedly asks for changes.
3	Why validate the payload?	Inbound data is untrusted and may be malformed or malicious.
4	What is idempotency?	Duplicate delivery does not create duplicate side effects.
5	Why acknowledge quickly?	Reduce timeout/retry risk and separate receipt from long processing.
6	Should GPT verify a cryptographic signature?	No; application/platform code should.
7	What does HTTP 2xx usually mean?	Successful receipt/processing at HTTP level.
8	Why keep an event ID?	Deduplication, tracing, and auditability.
9	What is n8n Test URL for?	Development/testing of the webhook workflow.
10	When use Respond to Webhook?	When you need controlled response data/status/headers to caller.

21A. Classroom Exercises

Exercise 1 - Read a payload

Given a sample order.created JSON body, identify the event ID, business object, dynamic fields, and fields that must be validated.

Exercise 2 - Design idempotency

A payment provider retries the same payment.completed event three times. Design the database check that guarantees only one invoice is created.

Exercise 3 - Choose the architecture

For each scenario choose API, polling, webhook-synchronous, or webhook-asynchronous: shipment update; nightly reconciliation; live AI endpoint; periodic inventory check.

Exercise 4 - Security review

Review a webhook design that stores the URL secret in source code, logs the entire body, accepts any event type, and issues refunds directly. Identify at least five problems.

Exercise 5 - n8n routing

Design an n8n Switch branch for lead.created, lead.updated, and lead.deleted. Unknown events should be logged and safely ignored.

21B. Homework

- 1. Build an n8n Webhook workflow and successfully send it a JSON POST request.
- 2. Add one authentication mechanism supported by your sender or test client.
- 3. Normalize inbound JSON into an internal schema.
- 4. Create an event_id deduplication check using a small database/data store.
- 5. Add GPT structured classification for a free-text field.
- 6. Create deterministic routing rules after the AI result.
- 7. Add a failure path for downstream API errors.
- 8. Create at least 12 webhook test cases, including duplicates and malicious/unexpected input.
- 9. Draw the architecture for fast acknowledgment plus background processing.
- 10. Write a one-page client proposal describing the business value of replacing polling with event-driven automation.

Homework deliverable

Submit: workflow screenshot, sample payload, response example, event schema, security notes, idempotency design, test table, and one architecture diagram.

22. Interview and Outsourcing Preparation

Interview questions and model answers

Question	Strong answer
What is a webhook?	An event-driven HTTP callback that notifies my endpoint when a subscribed event occurs.
How do you secure one?	HTTPS, provider auth/signature verification, secret management, validation, replay protection, least privilege.
How do you avoid duplicates?	Persist a stable event ID/idempotency key and make side effects conditional on first processing.
Why return quickly?	Reduce delivery timeout/retry risk; queue longer work when the caller only needs acknowledgment.
Webhook vs API?	Webhook is pushed by event source; API call is initiated by the consumer. They are usually used together.
How would n8n fit?	Webhook triggers the workflow, nodes validate/route, AI interprets text, integrations perform controlled actions, logging captures execution.

Client discovery questions

- Which system produces the event?
- Which exact event types are required?
- Does the provider retry failed webhooks?
- Does it provide event IDs?
- How does it sign/authenticate requests?
- How quickly must we acknowledge?
- Can processing run asynchronously?
- Which downstream actions are read-only vs write actions?
- What must be idempotent?
- What data is sensitive?
- How will failures be replayed?
- What monitoring and alerting are required?

Portfolio positioning

Present the project as Event-Driven AI Lead Operations System, not merely "an n8n webhook." Show the business event, security boundary, AI interpretation, deterministic controls, integrations, error handling, and measurable outcome.

23. Day 23 Checklist and Day 24 Preview

Day 23 mastery checklist

- I can explain webhook vs API vs polling.
- I can read a webhook request.
- I can receive JSON in Python or n8n.
- I know why authentication/signature verification belongs before AI.
- I can design idempotent event processing.
- I can separate acknowledgment from long-running business processing.
- I understand n8n Test vs Production webhook URLs.
- I can route event types safely.
- I can add GPT after validation for language interpretation.
- I can test duplicates, retries, invalid inputs, and downstream failures.

Golden formula

```
Reliable Real-Time Automation
=
Webhook Event
+ Sender Verification
+ Schema Validation
+ Idempotency
+ Fast Acknowledgment
+ Controlled Processing
+ AI Where Useful
+ Business Rules
+ Observability
```

Day 24 Preview - Introduction to n8n

Day 24 will turn these concepts into workflow engineering. Students will learn n8n workflow structure, triggers, nodes, expressions, data items, credentials, branching, loops, error workflows, executions, and how to assemble reusable AI automation workflows.

```
TRIGGER
|
v
DATA TRANSFORMATION
|
v
AI / API / LOGIC
|
v
BRANCH / LOOP
|
v
BUSINESS ACTION
|
v
LOG / RESPONSE
```

References and Further Reading

The current-product details in this lecture were checked against official documentation on August 19, 2026.

Official sources

[1] n8n - Webhook node

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>

[2] n8n - Webhook workflow development

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/workflow-development/>

[3] n8n - Respond to Webhook node

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtowehook/>

[4] OpenAI - Webhooks guide

<https://developers.openai.com/api/docs/guides/webhooks>

Final Day 23 takeaway

PRODUCTION MINDSET

The webhook endpoint is a trust boundary. Verify the sender, validate the event, deduplicate before side effects, acknowledge appropriately, and make AI one controlled component inside a reliable business workflow.